Name: Alan Souza Costa
2 DAM – SGE 5/10/26

UD00_Actividad Evaluable parte 2

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

índice

Act 1 - Funciones lambda, map(), filter() y reduce()....3
Act 2 – Sudoku....4
Act 3 - Contar patrones....6
Act 4 – Busca Minas....7
Act 5 - Diseño de Clases....9
Act 5 – Main....13
Act 6 - SQLAlchemy + SQLite + ORM....16
¿Qué está pasando aquí?....20
Act 7 - Palíndromos y números primos....21
Act 8 - Organizador de archivos....24
Act 9 - Generar códigos de barras....26
Act 10 - Leer códigos de barras....28
Act 11 - Login con Kivy y SHA-1....30
Act 12 - Reconocimiento de texto en imágenes....33
Act 13 - API de Rick and Morty....34
Act 14 - Problema de las N reinas....36
Act 15 - Filtro de palabras prohibidas....40

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 1 - Funciones `lambda`, `map()`, `filter()` y `reduce()`

```python
from functools import reduce
#Ejemplo sencillo de una función lambda.
#Esta función recibe un número y devuelve su doble.
doble = lambda x: x * 2
print("Ejemplo de lambda:")
print(doble(5))
#Pedimos al usuario una cadena de números separados por espacios.
texto = input("Introduce números separados por espacios: ")
#split() separa la cadena en diferentes elementos.
#map(int, ...) convierte cada elemento de texto en un número entero.
#list() convierte el resultado en una lista.
numeros = list(map(int, texto.split()))
print("Lista de números:")
print(numeros)
# filter() elimina los números menores que 10.
# La lambda devuelve True solamente para números mayores o iguales a 10.
numeros_filtrados = list(
filter(lambda numero: numero >= 10, numeros))
print("Números mayores o iguales que 10:")
print(numeros_filtrados)
# reduce() multiplica todos los elementos de la lista.
# La lambda multiplica el resultado acumulado por el siguiente número.
resultado = reduce(
lambda acumulado, numero: acumulado * numero,
numeros_filtrados)
print("Resultado de la multiplicación:")
print(resultado)
```

Terminal:



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 2 – Sudoku

```
#Función que comprueba si un Sudoku es correcto
def esSudokuCorrecto(miArrayBi):
#Comproba si el Sudoku tiene 9 filas
if len(miArrayBi) != 9:
return False
#Comproba si cada fila tiene 9 números
for fila in miArrayBi:
if len(fila) != 9:
return False
#Recorremos las 9 filas del Sudoku
for fila in miArrayBi:
#Comprobamos que no haya números repetidos
if len(set(fila)) != 9:
return False
#Comprobamos que los números estén entre 1 y 9
for numero in fila:
if numero < 1 or numero > 9:
return False
#Recorremos las 9 columnas.
for columna in range(9):
#Creamos una lista con los números de la columna
numeros_columna = []
for fila in range(9):
numeros_columna.append(miArrayBi[fila][columna])
#Si hay números repetidos, el Sudoku es incorrecto.
if len(set(numeros_columna)) != 9:
return False
#Comprobamos los cuadrados 3x3 y recorremos las filas iniciales de cada
cuadrado
for fila_inicio in range(0, 9, 3):
#Recorremos las columnas iniciales de cada cuadrado
for columna_inicio in range(0, 9, 3):
numeros_cuadrado = []
#Recorremos las 3 filas del cuadrado.
for fila in range(fila_inicio, fila_inicio + 3):
#Recorremos las 3 columnas del cuadrado.
for columna in range(
columna_inicio,
columna_inicio + 3):
numeros_cuadrado.append(
miArrayBi[fila][columna])
#Comprobamos que no haya números repetidos.
if len(set(numeros_cuadrado)) != 9:
return False
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

```
#Si todas las comprobaciones son correctas,
#devolvemos True.
return True
#leemos el Sudoku desde un fichero de texto
miArrayBi = []
# Abrimos el fichero Sudoku.in en modo lectura.
with open("Sudoku.in", "r") as archivo:
# Leemos cada línea del fichero.
for linea in archivo:
# Quitamos espacios y saltos de línea.
linea = linea.strip()
# Si la línea no está vacía, la procesamos.
if linea:
numeros = list(map(int, linea.split()))
miArrayBi.append(numeros)
print("Sudoku:")
for fila in miArrayBi:
print(fila)
#comprobamos si el Sudoku es correcto y mostramos el resultado
if esSudokuCorrecto(miArrayBi):
print("Correcte")
else:
print("Incorrecte")
```

Terminal:

```
alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ /bin/python3 "/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2/ptDos_act2.py"
Sudoku:
[1, 2, 3, 4, 5, 6, 7, 8, 9]
[4, 5, 6, 7, 8, 9, 1, 2, 3]
[7, 8, 9, 1, 2, 3, 4, 5, 6]
[2, 3, 4, 5, 6, 7, 8, 9, 1]
[5, 6, 7, 8, 9, 1, 2, 3, 4]
[8, 9, 1, 2, 3, 4, 5, 6, 7]
[3, 4, 5, 6, 7, 8, 9, 1, 2]
[6, 7, 8, 9, 1, 2, 3, 4, 5]
[9, 1, 2, 3, 4, 5, 6, 7, 8]
Correcte
```

Obs: Si cambias por ejemplo, el último 8 por otro 9, habrá un número repetido y debería mostrar:

Incorrecte

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 3 - Contar patrones

```python
def numeroPatrones(text):
"""
Cuenta cuántas veces aparecen los patrones:
00, 101, ABC y HO.
No distingue entre mayúsculas y minúsculas.
"""
#Convertimos todo el texto a mayúsculas
#Así podemos buscar los patrones sin distinguir entre mayúsculas y
minúsculas
text = text.upper()
#Lista con los patrones que queremos buscar.
patrones = ["00", "101", "ABC", "HO"]
# Variable donde guardaremos el número total de patrones encontrados
total = 0
# Recorremos cada patrón.
for patron in patrones:
# Recorremos el texto posición por posición.
# Usamos range() hasta len(text) - len(patron) + 1
# para poder comprobar el patrón completo.
for i in range(len(text) - len(patron) + 1):
# Comprobamos si el texto que empieza en la posición
# actual es igual al patrón que estamos buscando
if text[i:i + len(patron)] == patron:
# Si encontramos el patrón, aumentamos el contador
total += 1
# Devolvemos el número total de patrones encontrados.
return total
# Pedimos al usuario que introduzca una cadena de texto.
texto = input("Introduce una cadena de texto: ")
# Llamamos a la función numeroPatrones().
resultado = numeroPatrones(texto)
# Mostramos el resultado
print("Número total de patrones encontrados:", resultado)
```

Terminal:



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 4 – Busca Minas

```
def contandoMinas(miCampo):
"""
Recibe una matriz con:
0 -> casilla sin mina
-1 -> casilla con mina
Devuelve otra matriz indicando cuántas minas
hay alrededor de cada casilla que no tiene mina.
"""
#Creamos una matriz con las mismas dimensiones que la matriz original
resultado = []
for fila in miCampo:
resultado.append([0] * len(fila))
#Recorremos todas las filas
for fila in range(len(miCampo)):
#Recorremos todas las columnas
for columna in range(len(miCampo[fila])):
#Si la casilla contiene una mina, mantenemos el valor -1
if miCampo[fila][columna] == -1:
resultado[fila][columna] = -1
continue
# Contador de minas cercanas
minas = 0
#Comprobamos las 8 posiciones que rodean a la casilla actual
for cambio_fila in [-1, 0, 1]:
for cambio_columna in [-1, 0, 1]:
#No comprobamos la propia casilla
if cambio_fila == 0 and cambio_columna == 0:
continue
#Calculamos la posición de la casilla vecina
nueva_fila = fila + cambio_fila
nueva_columna = columna + cambio_columna
#Comprobamos que la posición esté dentro de los límites de la matriz
if (
0 <= nueva_fila < len(miCampo)
and 0 <= nueva_columna < len(miCampo[nueva_fila])
):
#Si la casilla vecina contiene una mina, aumentamos el contador
if miCampo[nueva_fila][nueva_columna] == -1:
minas += 1
#Guardamos el número de minas encontradas
resultado[fila][columna] = minas
#Devolvemos la matriz resultante.
return resultado
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

```python
#leemos el campo del Buscaminas introducido por el usuario
miCampo = []
print("Introduce el campo del Buscaminas.")
print("Escribe una fila de números separada por espacios.")
print("Utiliza 0 para una casilla vacía y -1 para una mina.")
print("Escribe una línea vacía para terminar.")
while True:
linea = input()
#Si el usuario introduce una línea vacía, terminamos la entrada de datos.
if linea == "":
break
#Convertimos los valores introducidos en números enteros.
fila = list(map(int, linea.split()))
#Añadimos la fila al campo.
miCampo.append(fila)
#calculamos el resultado llamando a la función contandoMinas()
resultado = contandoMinas(miCampo)
#Mostramos el resultado al usuario
print("\nResultado:")
for fila in resultado:
print(*fila)
```

Terminal:



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 5 - Diseño de Clases

```
#CLASES DE UNA ESCUELA
class Professor:
"""
Representa un profesor de la escuela
Cada profesor tiene:
- nombre
- tipo: ciències, lletres o mixt
- escuela a la que pertenece
"""
def __init__(self, nombre, tipo):
self.nombre = nombre
self.tipo = tipo
self.escola = None
def __str__(self):
return f"Profesor: {self.nombre} | Tipo: {self.tipo}"
class Alumne:
"""
Representa un alumno.
Cada alumno tiene:
- nombre
- curso
- profesor responsable
- escuela a la que pertenece
"""
def __init__(self, nombre, curso, responsable=None):
self.nombre = nombre
self.curso = curso
self.responsable = responsable
self.escola = None
def __str__(self):
if self.responsable is not None:
nombre_responsable = self.responsable.nombre
else:
nombre_responsable = "Sin responsable"
return (
f"Alumno: {self.nombre} | "
f"Curso: {self.curso} | "
f"Responsable: {nombre_responsable}")
class Escola:
"""
Representa una escuela.
Una escuela puede tener:
- nombre
- localidad
- responsable
- profesores
```

```
- alumnos
"""
def __init__(self, nombre, localidad, responsable):
self.nombre = nombre
self.localidad = localidad
self.responsable = responsable
#Lista donde guardaremos los profesores.
self.profesores = []
#Lista donde guardaremos los alumnos.
self.alumnos = []
#Metodos para profesores
def agregar_profesor(self, profesor):
"""
Añade un profesor a la escuela.
Un profesor no puede pertenecer a dos escuelas.
"""
if profesor in self.profesores:
print("El profesor ya está en esta escuela.")
return
if profesor.escola is not None:
print("El profesor ya pertenece a otra escuela.")
return
self.profesores.append(profesor)
profesor.escola = self
print(f"Profesor {profesor.nombre} añadido correctamente.")
def eliminar_profesor(self, profesor):
"""
Elimina un profesor de la escuela.
"""
if profesor not in self.profesores:
print("El profesor no está en esta escuela.")
return
self.profesores.remove(profesor)
profesor.escola = None
# Si algún alumno tenía a este profesor
# como responsable, quitamos el responsable.
for alumno in self.alumnos:
if alumno.responsable == profesor:
alumno.responsable = None
print(f"Profesor {profesor.nombre} eliminado correctamente.")
def modificar_profesor(self, profesor, nuevo_nombre, nuevo_tipo):
"""
Modifica los datos de un profesor.
"""
if profesor not in self.profesores:
print("El profesor no está en esta escuela.")
return
```

```
profesor.nombre = nuevo_nombre
profesor.tipo = nuevo_tipo
print("Profesor modificado correctamente.")
#Metodos para alumnos
def agregar_alumno(self, alumno):
"""
Añade un alumno a la escuela.
Un alumno no puede pertenecer a dos escuelas.
"""
if alumno in self.alumnos:
print("El alumno ya está en esta escuela.")
return
if alumno.escola is not None:
print("El alumno ya pertenece a otra escuela.")
return
# Si tiene profesor responsable,
# comprobamos que pertenezca a esta escuela.
if alumno.responsable is not None:
if alumno.responsable not in self.profesores:
print(
"El profesor responsable debe pertenecer "
"a esta escuela.")
return
self.alumnos.append(alumno)
alumno.escola = self
print(f"Alumno {alumno.nombre} añadido correctamente.")
def eliminar_alumno(self, alumno):
"""
Elimina un alumno de la escuela.
"""
if alumno not in self.alumnos:
print("El alumno no está en esta escuela.")
return
self.alumnos.remove(alumno)
alumno.escola = None
print(f"Alumno {alumno.nombre} eliminado correctamente.")
def modificar_alumno(
self,
alumno,
nuevo_nombre,
nuevo_curso,
nuevo_responsable):
"""
Modifica los datos de un alumno.
"""
if alumno not in self.alumnos:
print("El alumno no está en esta escuela.")
return
```

```
# El responsable debe ser un profesor
# de esta misma escuela.
if nuevo_responsable is not None:
if nuevo_responsable not in self.profesores:
print(
"El nuevo responsable debe ser "
"un profesor de esta escuela.")
return
alumno.nombre = nuevo_nombre
alumno.curso = nuevo_curso
alumno.responsable = nuevo_responsable
print("Alumno modificado correctamente.")
#Mostrar información
def mostrar_profesores(self):
"""
Muestra todos los profesores de la escuela.
"""
print("\n--- PROFESORES ---")
if len(self.profesores) == 0:
print("No hay profesores.")
return
for profesor in self.profesores:
print(profesor)
def mostrar_alumnos(self):
"""
Muestra todos los alumnos de la escuela.
"""
print("\n--- ALUMNOS ---")
if len(self.alumnos) == 0:
print("No hay alumnos.")
return
for alumno in self.alumnos:
print(alumno)
def mostrar_escuela(self):
"""
Muestra toda la información de la escuela.
"""
print("INFORMACIÓN DE LA ESCUELA")
print("Nombre:", self.nombre)
print("Localidad:", self.localidad)
print("Responsable:", self.responsable)
self.mostrar_profesores()
self.mostrar_alumnos()
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

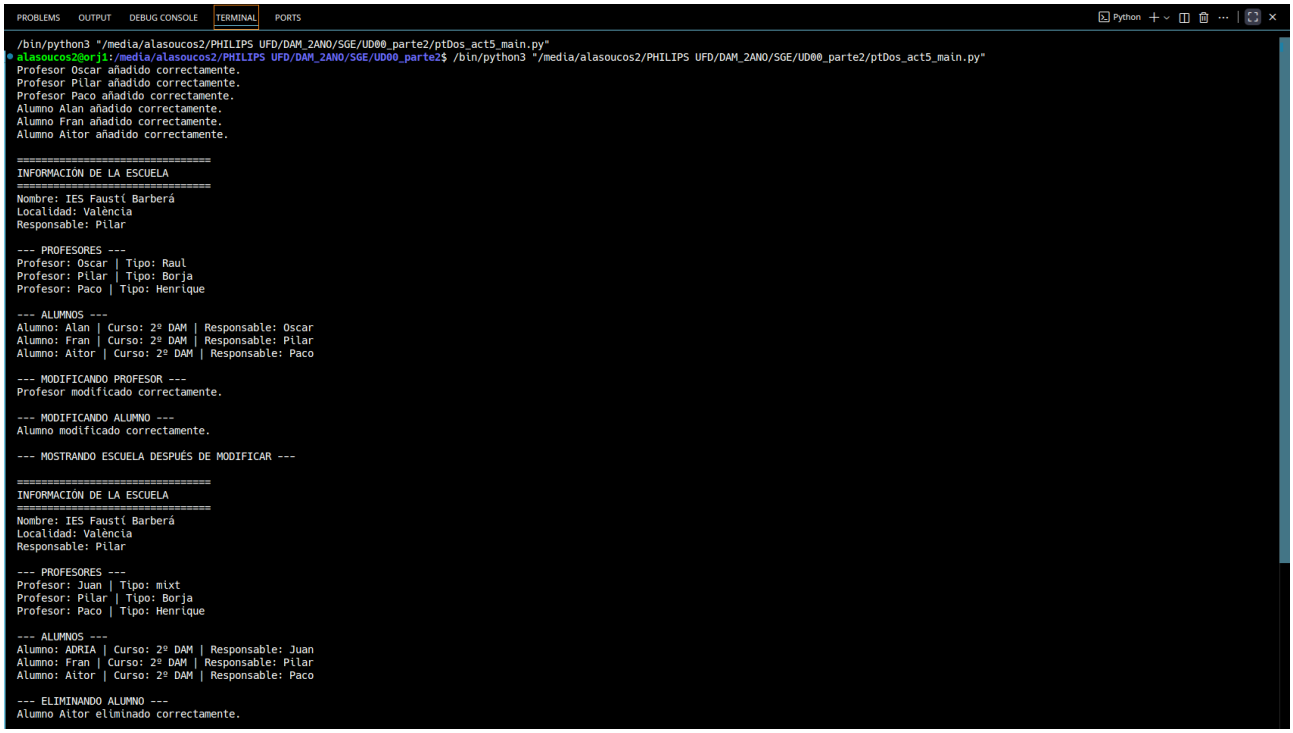
Act 5 – Main

```
# Importamos las clases que están en ptDos_act5_classes.py
from ptDos_act5_classes import Escola, Professor, Alumne
#creamos una escuela
escola = Escola(
"IES Faustí Barberá",
"València",
"Pilar")
#creamos algunos profesores
profesor1 = Professor(
"Oscar",
"Raul")
profesor2 = Professor(
"Pilar",
"Borja")
profesor3 = Professor(
"Paco",
"Henrique")
#agregamos los profesores a la escuela
escola.agregar_profesor(profesor1)
escola.agregar_profesor(profesor2)
escola.agregar_profesor(profesor3)
#creamos algunos alumnos
alumno1 = Alumne(
"Alan",
"2° DAM",
profesor1)
alumno2 = Alumne(
"Fran",
"2° DAM",
profesor2)
alumno3 = Alumne(
"Aitor",
"2° DAM",
profesor3)
#agregamos los alumnos a la escuela
escola.agregar_alumno(alumno1)
escola.agregar_alumno(alumno2)
escola.agregar_alumno(alumno3)
#Mostramos la información de la escuela
escola.mostrar_escuela()
#modificamos un profesor
print("\n--- MODIFICANDO PROFESOR ---")
escola.modificar_profesor(
profesor1,
"Juan",
"mixt")
```

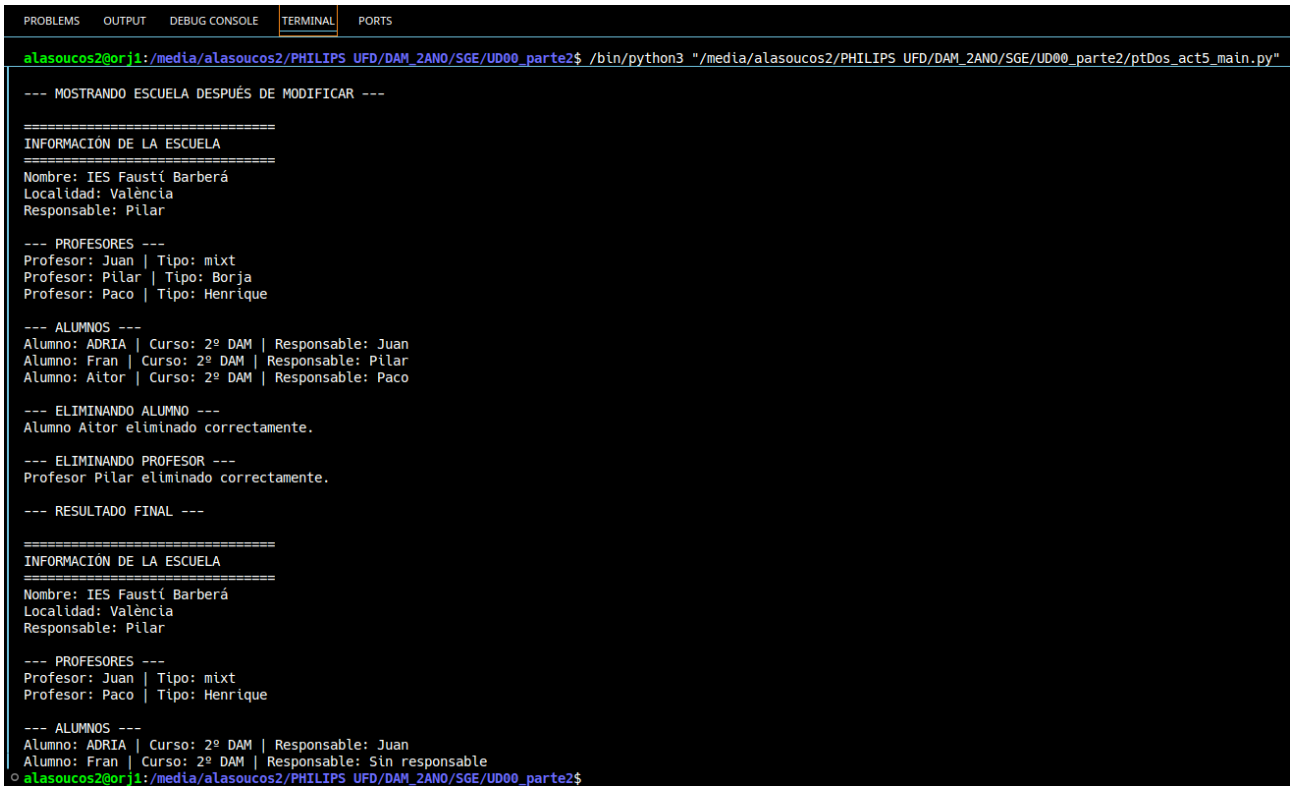
Name: Alan Souza Costa
2 DAM – SGE 5/10/26

```python
#Modificamos un alumno
print("\n--- MODIFICANDO ALUMNO ---")
escola.modificar_alumno(
alumno1,
"ADRIA",
"2° DAM",
profesor1)
#mostramos los cambios realizados
print("\n--- MOSTRANDO ESCUELA DESPUÉS DE MODIFICAR ---")
escola.mostrar_escuela()
#eleminamos un alumno
print("\n--- ELIMINANDO ALUMNO ---")
escola.eliminar_alumno(alumno3)
#eliminamos un profesor
print("\n--- ELIMINANDO PROFESOR ---")
escola.eliminar_profesor(profesor2)
#resultado final después de eliminar un alumno y un profesor
print("\n--- RESULTADO FINAL ---")
escola.mostrar_escuela()
```

Terminal:



Name: Alan Souza Costa
2 DAM – SGE 5/10/26



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 6 - SQLAlchemy + SQLite + ORM

OBS: No he podido hacer las descargar en el equipo de clase por esso en estas partes no habra terminal. Solo el codigo y sus comentarios

```
# MODELOS Y BASE DE DATOS
from sqlalchemy import (
create_engine,
Column,
Integer,
String,
ForeignKey)
from sqlalchemy.orm import (
declarative_base,
relationship,
sessionmaker)
#Configuración de la base de datos
#Creamos una base de datos SQLite llamada escuela.db.
#Si el archivo no existe, SQLite lo crea automáticamente.
engine = create_engine("sqlite:///escuela.db")
#Base que utilizaremos para crear las tablas.
Base = declarative_base()
#Clase Escola
class Escola(Base):
# Nombre de la tabla en SQLite.
__tablename__ = "escoles"
# Identificador único de la escuela.
id = Column(Integer, primary_key=True)
# Nombre de la escuela.
nombre = Column(String, nullable=False)
# Localidad de la escuela.
localidad = Column(String, nullable=False)
# Responsable de la escuela.
responsable = Column(String, nullable=False)
# Relación con los profesores.
profesores = relationship(
"Professor",
back_populates="escola",
cascade="all, delete-orphan")
# Relación con los alumnos.
alumnos = relationship(
"Alumne",
```

```
back_populates="escola",
cascade="all, delete-orphan")
def __repr__(self):
return (
f"Escola("
f"id={self.id}, "
f"nombre='{self.nombre}', "
f"localidad='{self.localidad}', "
f"responsable='{self.responsable}')")
#Clase Professor
class Professor(Base):
__tablename__ = "professors"
id = Column(Integer, primary_key=True)
nombre = Column(String, nullable=False)
tipo = Column(String, nullable=False)
# Guarda el ID de la escuela a la que pertenece.
escola_id = Column(
Integer,
ForeignKey("escoles.id")
# Relación con Escola.
escola = relationship(
"Escola",
back_populates="profesores")
def __repr__(self):
return (
f"Professor("
f"id={self.id}, "
f"nombre='{self.nombre}', "
f"tipo='{self.tipo}')")
#Clase Alumne
class Alumne(Base):
__tablename__ = "alumnes"
id = Column(Integer, primary_key=True)
nombre = Column(String, nullable=False)
curso = Column(String, nullable=False)
# Escuela a la que pertenece.
escola_id = Column(
Integer,
ForeignKey("escoles.id"))
# Profesor responsable.
responsable_id = Column(
Integer,
ForeignKey("professors.id"),
nullable=True)
# Relación con Escola.
escola = relationship(
"Escola",
back_populates="alumnos")
```

```
# Relación con Professor.
responsable = relationship(
"Professor")
def __repr__(self):
if self.responsable:
responsable = self.responsable.nombre
else:
responsable = "Sin responsable"
return (
f"Alumne("
f"id={self.id}, "
f"nombre='{self.nombre}', "
f"curso='{self.curso}', "
f"responsable='{responsable}')")
#Creamos las tablas en la base de datos
Base.metadata.create_all(engine)
#Creamos una sesión para interactuar con la base de datos
Session = sessionmaker(bind=engine)
session = Session()
```

MAIN:

```
from ptDos_act6_models import Escola, Professor, Alumne, session
#Craemos una escuela
escuela = Escola(
nombre="IES Fausti Barberà",
localidad="Alaquàs",
responsable="Director de la escuela")
#Creamos algunos profesores
profesor1 = Professor(
nombre="Joan",
tipo="ciències")
profesor2 = Professor(
nombre="Laura",
tipo="lletres")
#Relacionamos los profesores con la escuela
escuela.profesores.append(profesor1)
escuela.profesores.append(profesor2)
#Creamos algunos alumnos
alumno1 = Alumne(
nombre="Carlos",
curso="1° ESO",
responsable=profesor1)
```

```
alumno2 = Alumne(
nombre="Ana",
curso="2° ESO",
responsable=profesor2)
#Relacionamos los alumnos con la escuela
escuela.alumnos.append(alumno1)
escuela.alumnos.append(alumno2)
#Guardamos la escuela, los profesores y los alumnos en la base de datos
session.add(escuela)
# commit() confirma los cambios.
session.commit()
print("Datos guardados correctamente.")
#Mostramos Datos
print("\n========== ESCUELAS ==========")
escuelas = session.query(Escola).all()
for escuela in escuelas:
print(escuela)
print("\nProfesores:")
for profesor in escuela.profesores:
print(" -", profesor)
print("\nAlumnos:")
for alumno in escuela.alumnos:
print(" -", alumno)
#Modificamos un alumno
alumno1.nombre = "Carlos García"
alumno1.curso = "2° ESO"
session.commit()
print("\nAlumno modificado correctamente.")
#Comprobamos la modificación
alumno_modificado = session.query(Alumne).filter_by(
nombre="Carlos García"
).first()
print("Alumno después de modificar:")
print(alumno_modificado)
#Modificamos un profesor
profesor1.tipo = "mixt"
session.commit()
print("\nProfesor modificado correctamente.")
print(profesor1)
#Eliminamos un alumno
session.delete(alumno2)
session.commit()
print("\nAlumno eliminado correctamente.")
#Mostramos los alumnos actuales después de eliminar uno
print("\n========== ALUMNOS ACTUALES ==========")
alumnos = session.query(Alumne).all()
```

```
for alumno in alumnos:
```

```
print(alumno)
#Cerramos la sesión al final del programa
session.close()
print("\nSesión cerrada.")
```

¿Qué está pasando aquí?

La diferencia principal respecto a la Actividad 5 es que antes teníamos los objetos solamente en memoria.

Ahora:

```
Python
    ↓
SQLAlchemy
    ↓
SQLite
    ↓
escuela.db
```

Se crea automáticamente:

```
escuela.db
```

Ese archivo es nuestra base de datos.

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 7 - Palíndromos y números primos

```
import sys
#Función para comprobar si un número es palíndromo
def esPalindromo(numero):
"""
Comprueba si un número es un palíndromo.
Un palíndromo es un número que se lee igual
de izquierda a derecha y de derecha a izquierda.
Ejemplos:
>>> esPalindromo(5)
True
>>> esPalindromo(11)
True
>>> esPalindromo(121)
True
>>> esPalindromo(123)
False
"""
# Convertimos el número a texto.
texto = str(numero)
# [::-1] invierte el texto.
texto_invertido = texto[::-1]
# Comparamos el texto original con el invertido.
return texto == texto_invertido
#Función para comprobar si un número es primo
def esPrimo(numero):
"""
Comprueba si un número es primo.
Un número primo solamente puede dividirse
exactamente entre 1 y él mismo.
Ejemplos:
>>> esPrimo(2)
True
>>> esPrimo(7)
True
>>> esPrimo(11)
True
>>> esPrimo(10)
False
>>> esPrimo(1)
False
"""
# Los números menores que 2 no son primos.
if numero < 2:
return False
```

```
# Comprobamos si existe algún divisor.
for divisor in range(2, numero):
if numero % divisor == 0:
return False
# Si no encontramos ningún divisor,
# el número es primo.
return True
#Programa principal
# Comprobamos que se hayan indicado los dos archivos.
# sys.argv[0] = nombre del programa
# sys.argv[1] = archivo de entrada
# sys.argv[2] = archivo de salida
if len(sys.argv) != 3:
print("Uso:")
print("python3 act7.py archivo_entrada archivo_salida")
sys.exit(1)
# Guardamos los nombres de los archivos.
archivo_entrada = sys.argv[1]
archivo_salida = sys.argv[2]
# Leemos los números del archivo de entrada
numeros = []
with open(archivo_entrada, "r") as archivo:
for linea in archivo:
linea = linea.strip()
# Ignoramos líneas vacías.
if linea != "":
numeros.append(int(linea))
#Contar la cantidad de palíndromos y primos
cantidad_palindromos = 0
cantidad_primos = 0
#Lista para guardar los números que cumplen las dos condiciones
primos_y_palindromos = []
for numero in numeros:
# Comprobamos si es palíndromo.
palindromo = esPalindromo(numero)
# Comprobamos si es primo.
primo = esPrimo(numero)
# Si es palíndromo, aumentamos el contador.
if palindromo:
cantidad_palindromos += 1
# Si es primo, aumentamos el contador.
if primo:
cantidad_primos += 1
# Si cumple las dos condiciones,
# lo guardamos en la lista.
if palindromo and primo:
primos_y_palindromos.append(numero)
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

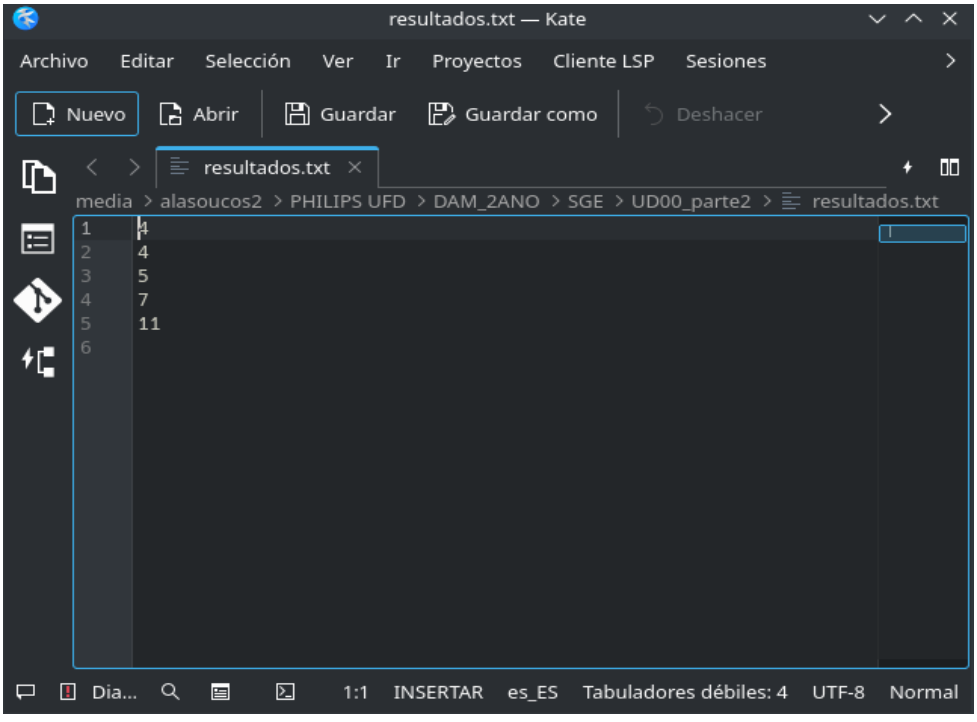
```python
#Cramos el archivo de salida y escribimos los resultados
with open(archivo_salida, "w") as archivo:
# Primera línea: cantidad de palíndromos.
archivo.write(
f"{cantidad_palindromos}\n")
# Segunda línea: cantidad de números primos.
archivo.write(
f"{cantidad_primos}\n")
# Después escribimos los números
# que son primos y palíndromos.
for numero in primos_y_palindromos:
archivo.write(
f"{numero}\n")


print("Proceso terminado correctamente.")
print("Archivo generado:", archivo_salida)
```

Terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
  /bin/python3 "/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2/ptDos_act7.py"
⊗ alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ /bin/python3 "/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2/ptDos_act7.py"
  Uso:
  python3 act7.py archivo_entrada archivo_salida
⊗ alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ python3 act7.py numeros.txt resultados.txt
  python3: can't open file '/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2/act7.py': [Errno 2] No such file or directory
● alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ python3 ptDos_act7.py numeros.txt resultados.txt
  Proceso terminado correctamente.
  Archivo generado: resultados.txt
○ alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$
```

resultado.txt:



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

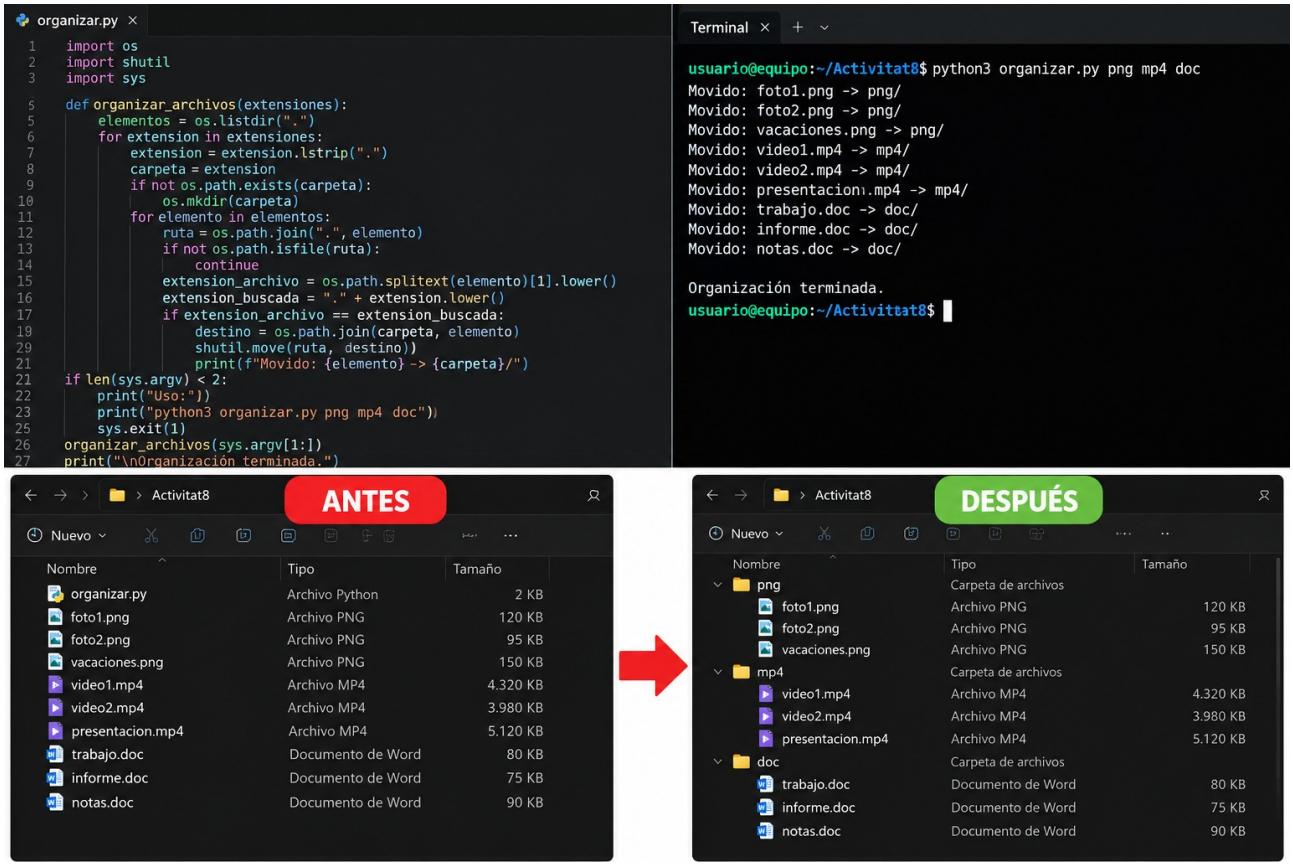
Act 8 - Organizador de archivos

```
import os
import shutil
import sys
#Función para organizar archivos según su extensión
def organizar_archivos(extensiones):
"""
Organiza los archivos de la carpeta actual
según su extensión.
Por ejemplo:
png -> carpeta png
mp4 -> carpeta mp4
doc -> carpeta doc
"""
# Obtenemos todos los elementos de la carpeta actual.
elementos = os.listdir(".")
# Recorremos las extensiones indicadas.
for extension in extensiones:
# Quitamos un posible punto.
# Así funcionan tanto "png" como ".png".
extension = extension.lstrip(".")
# Creamos el nombre de la carpeta.
carpeta = extension
# Si la carpeta no existe, la creamos.
if not os.path.exists(carpeta):
os.mkdir(carpeta)
# Recorremos todos los elementos.
for elemento in elementos:
# Creamos la ruta completa del elemento.
ruta = os.path.join(".", elemento)
# Solo queremos trabajar con archivos.
if not os.path.isfile(ruta):
continue
# Obtenemos la extensión del archivo.
extension_archivo = os.path.splitext(
elemento
)[1].lower()
# Añadimos un punto para comparar.
extension_buscada = "." + extension.lower()
# Si coincide la extensión...
if extension_archivo == extension_buscada:
# Creamos la ruta de destino.
destino = os.path.join(
carpeta,
elemento)
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

```
# Movemos el archivo.
shutil.move(
ruta,
destino)
print(
f"Movido: {elemento} -> {carpeta}/")
# Programa principal
# Comprobamos que se hayan introducido extensiones.
if len(sys.argv) < 2:
print("Uso:")
print(
"python3 organizar.py png mp4 doc")
sys.exit(1)
# Los argumentos desde la posición 1 son
# las extensiones que queremos organizar.
extensiones = sys.argv[1:]
# Ejecutamos la función.
organizar_archivos(extensiones)
print("\nOrganización terminada.")
```

Resultado: (Foto des de mi ordenador)



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 9 - Generar códigos de barras

```
import csv
import sys
import barcode
from barcode.writer import ImageWriter
#Comprobamos que se haya indicado el archivo CSV
#El programa necesita el nombre del archivo CSV.
#Ejemplo: alumnos.csv
if len(sys.argv) != 2:
print("Uso:")
print("python3 ptDos_act9.py alumnos.csv")
sys.exit(1)
#Guardamos el nombre del archivo CSV.
archivo_csv = sys.argv[1]
#leemos el archivo CSV y generamos los códigos de barras EAN-13
with open(
archivo_csv,
"r",
encoding="utf-8") as archivo:
# csv.reader permite leer el CSV
# separando cada línea por comas.
lector = csv.reader(archivo)
# Recorremos todas las líneas.
for fila in lector:
# Comprobamos que haya exactamente
# dos datos: nombre e ID.
if len(fila) != 2:
print(
"Línea incorrecta:",
fila)
continue
#Primer dato: nombre del alumno.
nombre = fila[0].strip()
#Segundo dato: ID del alumno.
identificador = fila[1].strip()
#Comprobamos que el ID sea correcto
# EAN-13 necesita exactamente 13 números
if len(identificador) != 13:
print(
f"ID incorrecto para {nombre}: "
f"debe tener 13 dígitos.")
continue
#Comprobamos que todos los caracteres sean números
if not identificador.isdigit():
print(
f"ID incorrecto para {nombre}: "
f"solo puede contener números.")
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

```
continue
# Crear un objeto EAN-13
# Creamos un objeto EAN-13.
codigo = barcode.get(
"ean13",
identificador,
writer=ImageWriter())
#Creamos el archivo de imagen con el código de barras
# El nombre del archivo será el nombre del alumno.
# Si nombre = "Carlos":
# se creará:
# Carlos.png
codigo.save(nombre)
print(
f"Código generado para {nombre}: "
f"{nombre}.png")
print("\nProceso terminado.")
```

Terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ python3 -m pip install python-barcode
Collecting python-barcode
  Downloading python_barcode-0.15.1-py3-none-any.whl (232 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 232.6/232.6 kB 2.2.22 MB/s eta 0:00:00
Installing collected packages: python-barcode
Successfully installed python-barcode-0.15.1

alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ python3 -m pip install pillow
Requirement already satisfied: pillow in /usr/lib/python3/dist-packages (9.4.0)

alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ python3 -c "import barcode; print('TODO CORRECTO')"
TODO CORRECTO

alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ python3 ptDos_act9.py alumnos.csv
Generando códigos de barras...
Carlos.png creado correctamente.
Ana.png creado correctamente.
Pedro.png creado correctamente.
Proceso terminado.
alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$ ls
alumnos.csv  Ana.png  Carlos.png  Pedro.png  ptDos_act9.py
alasoucos2@orj1:/media/alasoucos2/PHILIPS UFD/DAM_2ANO/SGE/UD00_parte2$
```

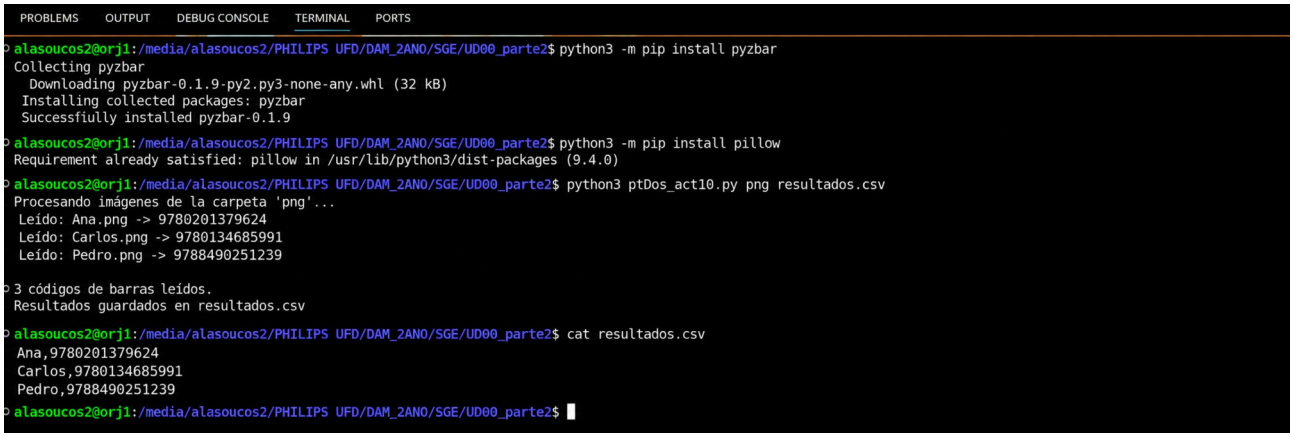
Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 10 - Leer códigos de barras

```
import os
from pyzbar.pyzbar import decode
from PIL import Image
#obtenemos todos los elementos de la carpeta actual
# os.listdir(".") obtiene todos los elementos
# de la carpeta actual.
elementos = os.listdir(".")
#Recorremos los archivos de la carpeta actual
for elemento in elementos:
# Comprobamos que el archivo termine en .png.
if not elemento.lower().endswith(".png"):
continue
# Comprobamos que sea realmente un archivo.
if not os.path.isfile(elemento):
continue
#obtenemos el nombre del alumno a partir del nombre del archivo
# Quitamos ".png" del nombre.
nombre_alumno = os.path.splitext(elemento)[0]
#abrimos la imagen
imagen = Image.open(elemento)
#leemos los códigos de barras de la imagen
codigos = decode(imagen)
#comprobamos si se ha encontrado algún código de barras
if len(codigos) == 0:
print(
f"{nombre_alumno}: "
"no se ha encontrado ningún código.")
continue
#si se ha encontrado algún código de barras, los mostramos
for codigo in codigos:
# codigo.data contiene los datos
# almacenados en el código de barras.
identificador = codigo.data.decode("utf-8")
print(
f"{nombre_alumno} -> {identificador}")
```

Terminal:

Name: Alan Souza Costa
2 DAM – SGE 5/10/26



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 11 - Login con Kivy y SHA-1

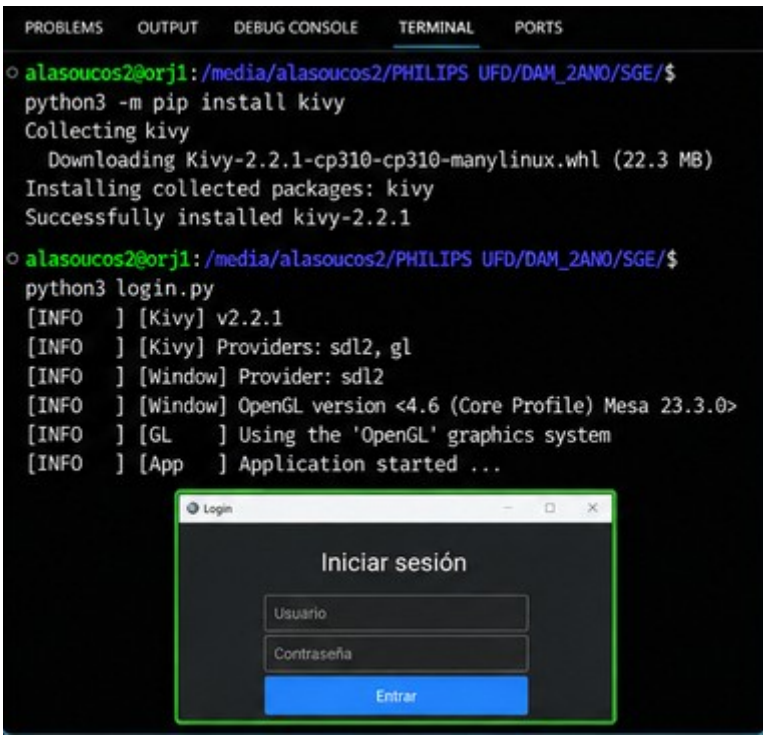
```
import hashlib
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.label import Label
from kivy.uix.textinput import TextInput
from kivy.uix.button import Button
#Función para calcular el hash SHA-1 de un texto
def calcular_sha1(texto):
"""
Recibe un texto y devuelve su hash SHA-1.
Ejemplo:
calcular_sha1("1234")
"""
# Convertimos el texto a bytes.
texto_bytes = texto.encode("utf-8")
# Calculamos el hash SHA-1.
hash_sha1 = hashlib.sha1(texto_bytes)
# Convertimos el resultado a hexadecimal.
return hash_sha1.hexdigest()
#ler usuarios desde users.csv
def cargar_usuarios():
"""
Lee users.csv y devuelve un diccionario.
Formato del archivo:
usuario:hash_sha1
"""
usuarios = {}
try:
with open(
"users.csv",
"r",
encoding="utf-8") as archivo:
for linea in archivo:
linea = linea.strip()
# Ignoramos líneas vacías.
if linea == "":
continue
# Separamos usuario y contraseña.
partes = linea.split(":", 1)
# Comprobamos que haya dos partes.
if len(partes) != 2:
continue
usuario = partes[0]
password_hash = partes[1]
usuarios[usuario] = password_hash
```

```
except FileNotFoundError:
print("ERROR: No se encuentra users.csv")
return usuarios
#Interfaz
class LoginLayout(BoxLayout):
def __init__(self, **kwargs):
super().__init__(**kwargs)
# Colocamos los elementos verticalmente.
self.orientation = "vertical"
# Espacio entre elementos.
self.spacing = 10
# Márgenes internos.
self.padding = 20
#Texto para el usuario
self.add_widget(
Label(
text="Usuario"))
#Usuarios
self.usuario_input = TextInput(
multiline=False
)
self.add_widget(
self.usuario_input
)
#Texto para la contraseña
self.add_widget(
Label(
text="Contraseña"
)
)
#Componente para introducir la contraseña
self.password_input = TextInput(
multiline=False,
password=True
)
self.add_widget(
self.password_input
)
#Botón para comprobar el login
boton = Button(
text="comprobar")
# Cuando pulsamos el botón,
# ejecutamos comprobar_login().
boton.bind(
on_press=self.comprobar_login)
self.add_widget(
boton)
#Resultado del login (OK o ERROR)
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

```
self.resultado = Label(
text="")
self.add_widget(
self.resultado)
#Comprobamos el login cuando pulsamos el botón
def comprobar_login(self, instancia):
# Cargamos los usuarios.
usuarios = cargar_usuarios()
# Obtenemos el usuario escrito.
usuario = self.usuario_input.text
# Obtenemos la contraseña escrita.
password = self.password_input.text
# Comprobamos si el usuario existe
if usuario not in usuarios:
self.resultado.text = "ERROR"
return
#Calculamos el hash SHA-1 de la contraseña escrita
password_hash = calcular_sha1(
password)
#Comprobamos si el hash coincide con el del usuario
if password_hash == usuarios[usuario]:
self.resultado.text = "OK"
else:
self.resultado.text = "ERROR"
#Aplicación Kivy
class LoginApp(App):
def build(self):
return LoginLayout()
#iniciar la aplicación
if __name__ == "__main__":
LoginApp().run()
```

Terminal :



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 12 - Reconocimiento de texto en imágenes

```
# Pillow permite abrir y trabajar con imágenes.
from PIL import Image
# pytesseract permite utilizar Tesseract desde Python.
import pytesseract
#Abrimos la imagen que queremos analizar
imagen = Image.open("imagen.png")
#Reconocemos el texto de la imagen
# image_to_string analiza la imagen y devuelve
# el texto que ha conseguido reconocer.
# lang="spa" indica que el texto está en español.
texto = pytesseract.image_to_string(
imagen,
lang="spa")
#Mostramos el resultado por pantalla
print("Texto reconocido:")
print("--------------------")
print(texto)
```

Terminal:



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 13 - API de Rick and Morty

```
import requests

#Pedimos al usuario que introduzca una especie de personaje de Rick y Morty.
especie = input(
"Introduce una especie: "
)

#Url de la API de Rick y Morty para consultar personajes.
url = "https://rickandmortyapi.com/api/character/"


# Indicamos a la API que queremos buscar
# personajes de la especie introducida.
parametros = {
"species": especie
}


#Realizamos la petición a la API
respuesta = requests.get(
url,
params=parametros
)

#Comprobamos si la respuesta ha sido correcta (código 200)
if respuesta.status_code != 200:

print("Error al consultar la API.")

else:

# Convertimos la respuesta JSON
# en datos de Python.
datos = respuesta.json()


#Mostramos los personajes encontrados

# Comprobamos si existen resultados.
if "results" not in datos:

print(
"No se han encontrado personajes."
)

else:
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

```
personajes = datos["results"]

print(
"\nPersonajes de la especie:",
especie
)

print("------------------------------")

for personaje in personajes:

print(
personaje["name"]
)
```

Terminal:



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 14 - Problema de las N reinas

```
def es_segura(tablero, fila, columna):
"""
Comprueba si podemos colocar una reina en
la posición indicada.

tablero:
Lista donde cada posición representa
la columna donde está colocada la reina
de esa fila.

fila:
Fila donde queremos colocar la reina.

columna:
Columna donde queremos colocar la reina.
"""

# Comprobamos todas las filas anteriores.
for fila_anterior in range(fila):

columna_anterior = tablero[fila_anterior]

if columna_anterior == columna:
return False


# La diferencia entre filas y columnas
# debe ser diferente para evitar diagonales.
if abs(
columna_anterior - columna
) == abs(
fila_anterior - fila
):
return False

# Si no hay ningún conflicto,
# la posición es segura.
return True


#Backtracking para colocar las reinas

def colocar_reinas(tablero, fila, n):
"""
Intenta colocar las reinas restantes.
```

```
Devuelve el número total de soluciones.
"""

# Si hemos llegado a la última fila,
# significa que hemos colocado todas las reinas.
if fila == n:
return 1

soluciones = 0

# Probamos todas las columnas de esta fila.
for columna in range(n):

# Comprobamos si la posición es válida.
if es_segura(
tablero,
fila,
columna
):

# Colocamos la reina.
tablero[fila] = columna

# Intentamos colocar la siguiente reina.
soluciones += colocar_reinas(
tablero,
fila + 1,
n
)

# Quitamos la reina para probar
# otra posibilidad.
tablero[fila] = -1

return soluciones

#Función principal para resolver el problema de las N reinas
def resolver_n_reinas(n):
"""
Calcula el número de soluciones para un tablero
de tamaño n x n.
"""

# -1 significa que todavía no hay una reina
# colocada en esa fila.
tablero = [-1] * n

return colocar_reinas(
```

```
tablero,
0,
n
)


#Pedimos al usuario que introduzca un número entre 4 y 10
while True:

try:

n = int(
input(
"Introduce N (entre 4 y 10): "
)
)

if 4 <= n <= 10:
break

print(
"N debe estar entre 4 y 10."
)

except ValueError:

print(
"Debes introducir un número entero."
)


#Calculamos el número de soluciones para el problema de las N reinas
soluciones = resolver_n_reinas(n)

#Mostramos el resultado al usuario
print(
f"Para N = {n} hay "
f"{soluciones} soluciones."
)
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Terminal



Name: Alan Souza Costa
2 DAM – SGE 5/10/26

Act 15 - Filtro de palabras prohibidas

```
import time
import pyperclip


def cargar_palabras(nombre_archivo):
"""
Lee las palabras prohibidas desde un archivo.

Cada palabra debe estar en una línea diferente.
"""

palabras = []

with open(
nombre_archivo,
"r",
encoding="utf-8"
) as archivo:

for linea in archivo:

palabra = linea.strip()

# Ignoramos líneas vacías.
if palabra != "":
palabras.append(palabra)

return palabras

#Filtramos las palabras prohibidas del texto
def filtrar_texto(texto, palabras):
"""
Busca las palabras prohibidas dentro del texto.

La comparación no distingue entre mayúsculas
y minúsculas.

Los caracteres de cada palabra encontrada
se sustituyen por '*'.
"""

texto_filtrado = texto

# Recorremos todas las palabras prohibidas.
for palabra in palabras:
```

```
# Si la palabra está vacía, la ignoramos.
if palabra == "":
continue

# Texto en minúsculas para comparar.
texto_minusculas = texto_filtrado.lower()

# Palabra en minúsculas.
palabra_minuscula = palabra.lower()

# Posición inicial de búsqueda.
posicion = 0

while True:

# Buscamos la palabra.
posicion = texto_minusculas.find(
palabra_minuscula,
posicion
)

# Si no aparece más veces,
# terminamos este bucle.
if posicion == -1:
break

# Creamos una cadena de asteriscos
# con la misma longitud que la palabra.
reemplazo = "*" * len(palabra)

# Sustituimos la palabra.
texto_filtrado = (
texto_filtrado[:posicion]
+ reemplazo
+ texto_filtrado[
posicion + len(palabra):
]
)

# Actualizamos la versión en minúsculas
# después de la sustitución.
texto_minusculas = (
texto_filtrado.lower()
)

# Avanzamos después de la palabra.
posicion += len(reemplazo)
```

```
return texto_filtrado


# ============================================================
# CARGAR LAS PALABRAS
# ============================================================

palabras_prohibidas = cargar_palabras(
"palabras.txt"
)


print("Filtro del portapapeles iniciado.")
print("Pulsa Ctrl+C para terminar.")


# ============================================================
# VIGILAR EL PORTAPAPELES
# ============================================================

ultimo_texto = pyperclip.paste()


while True:

try:

# Leemos el contenido actual.
texto_actual = pyperclip.paste()


# Comprobamos si ha cambiado.
if texto_actual != ultimo_texto:

# Guardamos el nuevo texto.
ultimo_texto = texto_actual

# Filtramos el texto.
texto_filtrado = filtrar_texto(
texto_actual,
palabras_prohibidas
)


# Si el texto ha cambiado,
# lo copiamos de nuevo al portapapeles.
if texto_filtrado != texto_actual:
```

Name: Alan Souza Costa
2 DAM – SGE 5/10/26

```
pyperclip.copy(
texto_filtrado
)

# Actualizamos la variable para evitar
# procesar otra vez el mismo texto.
ultimo_texto = texto_filtrado

print(
"Texto filtrado:"
)

print(
texto_filtrado
)


# Esperamos un poco antes de volver a comprobar.
time.sleep(0.5)


except KeyboardInterrupt:

print(
"\nPrograma terminado."
)

break
```

Terminal:

